

EasyNet-Cli



easynet

CLI + MCP server for **EasyNet Axon** — manage distributed devices, orchestrate abilities across edge nodes, and give Claude Code direct access to your fleet.

EASYNET	AXON	LICENSE	APACHE 2.0		
 RUST	1.75+	PLATFORM	MACOS LINUX	EAL	V0.1

What is this?

`easynet` is a single binary that turns a fleet of edge devices into a programmable network. It speaks to an **Axon Hub** — the coordination layer that routes ability invocations across devices behind NAT, without any of them needing a public IP.

Three things in one binary:

1. **CLI** — `easynet devices`, `easynet exec gpu-rig -- nvidia-smi`, `easynet deploy ./skill --to edge-cam`. Ten subcommands for real-time device and ability management.

2. **EAL compiler** — EAL (EasyNet Ability Language) is a DSL for distributed ability orchestration. Write a `.eal` file, the compiler infers a dependency DAG from variable references, partitions it into parallel phases, and compiles it to **Mission IR v2** — a serializable, backend-agnostic execution plan.
3. **MCP server** — `easynet mcp-server` exposes 11 Hub-level tools over stdio, so Claude Code or Codex can discover devices, deploy abilities, run commands, and orchestrate missions without leaving the IDE.

Why this matters: Most distributed execution tools force you to choose between a CLI (manual, one-shot) and a programmable API (code-heavy, no visibility). EAL sits in between — it's a language with compiler-level guarantees (type-safe variable references, cycle detection, phase-optimal parallelism) that produces an inspectable IR and a structured audit trail.

Install

```
cargo install --path .
```

Note: Requires a local or remote Axon runtime. The `easynet start` command auto-spawns one and joins the Hub.

Quick Start

Join a Hub

```
easynet start --hub axon://hub.easynet.run:50084 --tenant myteam --foreground
# ✓ Axon runtime started on http://127.0.0.1:50123
# ✓ Joined hub.easynet.run as alice-macbook
```

Explore the fleet

```
easynet devices
```

```
# |-----|-----|-----|-----|
```

#		NODE	STATE	OS	TRUST
#					
#	●	home-server	HEALTHY	linux/amd64	TRUSTED
#	●	gpu-rig	HEALTHY	linux/arm64	TRUSTED
#	○	rpi-lab	OFFLINE	linux/arm	PROBATION
#					

easynet abilities

#				
#	ABILITY	NODE	VERSION	STATUS
#				
#	photo.capture	edge-cam-01	1.2.0	ACTIVE
#	model.inference	gpu-rig	3.1.0	ACTIVE
#				

Remote execution

```
easynet exec gpu-rig -- nvidia-smi --query-gpu=name,memory.used --format=csv
# | tunnel via hub.easynet.run (E2E encrypted)
# | name, memory.used [MiB]
# | NVIDIA A100-SXM4-80GB, 12453 MiB
# | ✓ done
```

Deploy an ability

```
easynet deploy ./skills/photo-capture --to edge-cam-01
# | publishing photo.capture@1.2.0 ... ✓
# | installed (install_id: inst-7x8k)
# | ✓ activated – photo.capture is live
```

Run an EAL mission

```
easynet mission run examples/daily-report.eal
# | mission: daily-report (6 steps, 4 nodes, 3 phases)
# |
# | phase 0 (parallel):
# | [1/6] photo.capture      edge-cam-01    ✓ 1.2s → $photo
# | [2/6] config.fetch      coordinator ✓ 0.3s → $config
# | [3/6] metrics.ping      monitor    ✓ 0.1s
# |
# | phase 1:
# | [4/6] model.inference   gpu-rig    ✓ 3.8s → $result (← $photo, $config)
# |
```

```
# phase 2:
# [5/6] data.collect      home-server    ✓ 0.9s → $report (← $result)
#
# ✓ Mission completed — 6.3s across 4 nodes (3 phases)
```

EAL (EasyNet Ability Language)

EAL is a DSL for distributed ability orchestration. It separates **what to execute** from **how to schedule it** — you write the data flow, the compiler figures out the parallelism.

```
mission "daily-report" {
  // Phase 0: independent — dispatched in parallel
  let photo = call "photo.capture" on "edge-cam-01" with {
    resolution = "4k"
  } timeout 30

  let config = call "config.fetch" on "coordinator"

  // Phase 1: depends on photo + config — data-flow barrier
  let result = call "model.inference" on "gpu-rig" with {
    input = photo.output,
    model_config = config.output
  } timeout 120 retries 2 on_failure retry

  // Phase 2: depends on result
  let report = call "data.collect" on "home-server" with {
    data = result.output,
    template = "daily-report"
  }

  // Fire-and-forget (phase 0, no binding)
  call "metrics.ping" on "monitor" optional
}
```

Compiler pipeline

```
.eal source → Lexer → Parser → Analyzer → Planner → Mission IR v2 → Interpreter
```

Stage	Responsibility
Lexer	Tokenizes keywords, literals, identifiers, comments

Stage	Responsibility
Parser	Recursive descent → <code>EalProgram</code> AST
Analyzer	Symbol table, VarRef validation, cycle detection (DFS), retry policy enforcement
Planner	Topological layering → phase partitioning. Dependencies inferred from variable references
IR	Mission IR v2 — serializable JSON with <code>input_refs</code> + <code>output_binding</code> per step
Interpreter	Parallel dispatch (<code>std::thread::scope</code>), retry with exponential backoff, structured <code>ExecutionTrace</code>

Key design decisions

- **Dependencies are inferred, not declared.** Write `input = photo.output` and the compiler builds the DAG. No `depends_on` arrays.
- **Phase partitioning is optimal.** Steps land in the earliest possible phase where all dependencies are resolved. Independent steps always run in parallel.
- **Mission IR v2 is the contract.** The IR is serializable, inspectable (`--emit-ir`), and backend-agnostic. Today it runs on a client-side interpreter; tomorrow on server-side MissionControl v2.
- **Retry is deterministic.** Exponential backoff with SHA-256-based jitter seeded by `(step_id, attempt)` — reproducible across runs.
- **Every execution produces an audit trail.** `ExecutionTrace` captures per-step timestamps, result SHA-256 hashes, retry history, and phase structure. Output via `--trace` .

Inspection

```
# Compile to IR without executing
easynet mission run examples/diamond.eal --emit-ir

# Execute with full audit trace
easynet mission run examples/daily-report.eal --trace
```

Language Layering

AAL (Agent Assembly Language) goals, planning, memory, decisions		future – agent behavior
EAL (EasyNet Ability Language) call, let, data flow, failure policy compiles to Mission IR v2		this project
Mission IR v2 steps + input_refs + output_binding		serializable execution plan
Client Interp. (current)	MissionControl v2 (future) server-side, stateful, checkpoint, resume	
Axon Runtime + Federation + Hub		existing infrastructure

EAL describes **distributed ability execution**. AAL (future) will describe **agent behavior** — goals, planning, memory — and emit EAL programs as its execution substrate.

MCP Server

`easynet mcp-server` runs a Hub-level MCP server on stdio. Configure Claude Code:

```
{
  "mcpServers": {
    "easynet": {
      "command": "easynet",
      "args": ["mcp-server", "--tenant", "myteam"]
    }
  }
}
```

Available tools

Tool	Description
<code>hub_status</code>	Hub connection, node/ability counts
<code>list_devices</code>	All devices across federation
<code>get_device_detail</code>	Device info + installed abilities
<code>list_all_abilities</code>	Abilities across all nodes
<code>search_abilities</code>	Find by name pattern
<code>deploy_ability</code>	Publish → install → activate pipeline
<code>execute_command</code>	One-shot command on remote device
<code>invoke_ability</code>	Invoke ability on any federated node
<code>run_mission</code>	Compile + execute EAL program
<code>manage_device</code>	Drain / disconnect device
<code>uninstall_ability</code>	Remove ability from device

CLI Reference

```
easynet start    --hub <endpoint> [--tenant T] [--label L] [--token T] [--foreground]
easynet stop
easynet status
easynet devices  [--state online|offline] [--format table|json]
easynet abilities [--node N] [--format table|json]
easynet exec     <node> -- <command...>
easynet deploy   <path> --to <node>
easynet invoke   <node> <ability> [--args JSON]
easynet mission  run <file.eal> [--emit-ir] [--trace]
easynet mcp-server [--endpoint URL] [--tenant T]
```

Examples

See [examples/](#) for EAL programs:

File	Pattern	Phases
<code>hello.eal</code>	Single step	1
<code>parallel.eal</code>	Independent steps	1 (all parallel)
<code>pipeline.eal</code>	Linear chain $A \rightarrow B \rightarrow C$	3 (fully sequential)
<code>diamond.eal</code>	Diamond: $A \rightarrow B, A \rightarrow C, B+C \rightarrow D$	3 (B/C in phase 1)
<code>daily-report.eal</code>	Full multi-node orchestration	3 (4 parallel in phase 0)

License

Apache-2.0 — see [LICENSE](#).

Author

Silan Hu · silan.hu@u.nus.edu